

MODELADO NUMÉRICO Y VERIFICACIÓN DE LAS LEYES DE KEPLER MEDIANTE GRAVITACIÓN UNIVERSAL

Pre-proyecto STEAM

Grupo 6 — Taller 19

Edizon Alexander Mesa Leal — *Líder de Diseño Físico/Computacional*

Andrés Elías Roa Puentes — *Analista Matemático*

John Esteban Lievano Mendez — *Especialista en Toma y Análisis de Datos*

Samuel David Loaiza Ocampo — *Coordinador de Redacción y Presentación*

Proyecto académico de Física Mecánica

2026

Índice

1. Planteamiento del Problema y Justificación	2
2. Objetivos	2
2.1. Objetivo General	2
2.2. Objetivos Específicos	2
3. Fundamentación Física (Marco Teórico Preliminar)	3
3.1. Ley de Gravitación Universal	3
3.2. Segunda Ley de Newton	3
3.3. Energía Mecánica	3
3.4. Momento Angular	4
3.5. Integración Numérica	4
3.6. Leyes de Kepler	4
3.7. Diagrama de Cuerpo Libre Preliminar	4
4. Diseño Preliminar y Bocetos	4
4.1. Flujo de Datos	4
4.2. Estructura de Carpetas	5
4.3. Archivos y Responsabilidades	6
4.4. Tecnologías	6

5. Metodología de Medición y Plan de Pruebas	7
5.1. Variables	7
5.2. Escenarios de Prueba	7
5.3. Checklist de Implementación (Orden de construcción)	7
5.4. Consideraciones de Diseño Críticas	8
5.5. Incertidumbre y Error	9
6. Resultados Esperados (Hipótesis)	9
6.1. Predicciones Específicas	9
6.2. Gráficas Previstas	9
7. Lista de Materiales y Presupuesto	10
8. Análisis de Riesgos y Seguridad	11
8.1. Medidas Críticas	11
9. Cronograma de Actividades (Diagrama de Gantt)	11
10. Referencias	11

1. Planteamiento del Problema y Justificación

Predecir el movimiento orbital de un cuerpo celeste a partir de sus condiciones iniciales requiere relacionar una ley física continua con un procedimiento numérico capaz de aproximar su evolución paso a paso. Aunque la ley de gravitación universal permite calcular la fuerza entre dos masas, una computadora necesita transformar esa relación en posiciones y velocidades discretas.

El desafío técnico es construir un simulador que no solo produzca una trayectoria visualmente convincente, sino que también sea físicamente verificable. Un resultado gráfico atractivo no demuestra por sí solo la exactitud del modelo: la simulación debe mostrar estabilidad numérica y conservar, dentro de una tolerancia definida (ej. 1 %), magnitudes físicas fundamentales que deben permanecer constantes en un sistema ideal.

Desde la física mecánica, el proyecto aborda la Segunda Ley de Newton, la Ley de Gravitación Universal, la conservación de la energía mecánica, el momento angular y la dinámica orbital. Desde el enfoque STEAM/computacional, integra programación estructurada, métodos numéricos (Euler-Cromer y Verlet), visualización interactiva y evaluación objetiva del error numérico.

Aplicación: el modelo sirve como herramienta educativa para explorar cómo una pequeña modificación en la velocidad inicial puede cambiar el tipo de órbita. También ilustra un principio general de la simulación científica: separar el cálculo del fenómeno de su representación gráfica y validar el algoritmo con magnitudes independientes.

2. Objetivos

2.1. Objetivo General

Desarrollar un simulador computacional interactivo en Python que modele el movimiento orbital de uno o varios cuerpos bajo gravitación universal y que permita identificar trayectorias circulares, elípticas e hiperbólicas, verificando cuantitativamente la conservación de energía y momento angular y las tres leyes de Kepler.

2.2. Objetivos Específicos

- Definir una representación computacional de los cuerpos celestes mediante masa, posición, velocidad y nombre.
- Calcular la aceleración gravitacional entre múltiples cuerpos empleando la Ley de Gravitación Universal.
- Implementar los integradores Euler-Cromer y Verlet y comparar su estabilidad numéricamente y en términos de conservación de energía.
- Registrar posición, velocidad, energía mecánica y momento angular durante toda la simulación para validar el integrador.
- Construir pruebas controladas para generar órbitas circular, elíptica e hiperbólica a partir de diferentes velocidades iniciales.
- Verificar la Primera Ley de Kepler mediante el ajuste/inspección de la trayectoria y la localización del foco.

- Verificar la Segunda Ley de Kepler mediante el cálculo del área barrida en intervalos de tiempo iguales.
- Verificar la Tercera Ley de Kepler comparando el periodo orbital (T) con el semieje mayor (a) mediante la relación T^2/a^3 .
- Definir una tolerancia de error (por ejemplo, 1 %) como criterio objetivo inicial y reportar la desviación observada.
- Separar formalmente el motor de simulación física de la capa de renderizado visual para permitir miles de pasos de integración sin quedar limitado por la tasa de cuadros de la interfaz.

3. Fundamentación Física (Marco Teórico Preliminar)

3.1. Ley de Gravitación Universal

Dos masas m_1 y m_2 separadas por una distancia r se atraen con una fuerza cuyo módulo es proporcional al producto de las masas e inversamente proporcional al cuadrado de la distancia:

$$r = \sqrt{\frac{G \cdot m_1 \cdot m_2}{F}} \quad (1)$$

En forma vectorial, para calcular la aceleración de un cuerpo i producida por un cuerpo j , se empleará una expresión equivalente a:

$$\vec{a}_i = G \sum_{j \neq i} \frac{m_j (\vec{r}_j - \vec{r}_i)}{|\vec{r}_j - \vec{r}_i|^3} \quad (2)$$

La forma vectorial permite obtener simultáneamente dirección y magnitud.

3.2. Segunda Ley de Newton

La dinámica del cuerpo se obtiene de $\vec{F} = m\vec{a}$. En el modelo, la aceleración calculada por gravedad es la entrada principal de los integradores numéricos.

3.3. Energía Mecánica

Para un sistema ideal sin fuerzas disipativas, la energía mecánica total debe permanecer aproximadamente constante. En un caso de dos cuerpos se considerará la suma de energía cinética y potencial gravitacional:

$$E = K + U = \frac{1}{2}mv^2 - G\frac{Mm}{r} \quad (3)$$

La variación relativa de energía se utilizará como indicador de error numérico. Si la energía se aleja sistemáticamente de su valor inicial, será necesario revisar el paso dt , el integrador o el cálculo de gravedad.

3.4. Momento Angular

El momento angular respecto al origen se calculará mediante $\vec{L} = \vec{r} \times (m\vec{v})$:

$$L = r \times (mv) \quad (4)$$

Para un sistema aislado con fuerza gravitacional central, el momento angular total debe conservarse.

3.5. Integración Numérica

Euler-Cromer actualizará primero la velocidad y luego la posición usando el nuevo valor de velocidad. Es sencillo y suele ser más estable para problemas oscilatorios/orbitales que el Euler explícito. Verlet se implementará como segundo método para comparar precisión y conservación.

3.6. Leyes de Kepler

- **Primera ley:** Las órbitas ligadas son elipses con el cuerpo central situado en uno de sus focos.
- **Segunda ley:** El radio vector barre áreas iguales en tiempos iguales.
- **Tercera ley:** Para cuerpos que orbitan el mismo cuerpo central, T^2 es proporcional a a^3 .

Unidades: para reducir escalas extremas y mejorar la interpretación, se recomienda usar unidades normalizadas/astronómicas (AU, masas solares, años) en la interfaz y documentar claramente la convención adoptada. El código debe mantener una sola convención interna y no mezclar SI con unidades astronómicas sin una conversión explícita.

3.7. Diagrama de Cuerpo Libre Preliminar

El cuerpo orbitante m está sujeto únicamente a la atracción gravitacional central $\vec{F}_g$ ejercida por la masa central M .

4. Diseño Preliminar y Bocetos

4.1. Flujo de Datos

1. Usuario ingresa masas/velocidades iniciales $\rightarrow$ `ui/controls.py`
2. Se crean objetos Body $\rightarrow$ `physics/bodies.py`
3. Loop de simulación (`main.py`):
 - `gravity.py` calcula fuerza/aceleración entre todos los cuerpos
 - `integrators/*.py` usa esa aceleración para calcular nueva posición/velocidad (dt)
 - `conservation.py` registra energía y momento angular del nuevo estado

- Estado actualizado se guarda en un historial (lista/array)
- Cada N pasos, `render/scene3d.py` lee el historial y actualiza el gráfico 3D
- Al final (o en tiempo real), `render/plots.py` grafica energía/momento angular
- `kepler/kepler_laws.py` analiza el historial completo y confirma las 3 leyes

Punto crítico: El loop de física corre independiente del renderizado. No calcules física dentro de la función que dibuja. Si acoplas ambos, tu simulación se vuelve tan lenta como tu framerate de gráficos, y no puedes correr miles de pasos para verificar Kepler con precisión.

4.2. Estructura de Carpetas

```
orbital-simulator/
|-- src/
|   |-- physics/
|   |   |-- gravity.py
|   |   |-- bodies.py
|   |   |-- conservation.py
|   |-- integrators/
|   |   |-- euler_cromer.py
|   |   |-- verlet.py
|   |-- kepler/
|   |   |-- kepler_laws.py
|   |-- render/
|   |   |-- scene3d.py
|   |   |-- plots.py
|   |-- ui/
|   |   |-- controls.py
|   |-- main.py
|-- tests/
|   |-- test_gravity.py
|   |-- test_integrators.py
|   |-- test_conservation.py
|   |-- test_kepler.py
|-- data/
|   |-- presets.json
|-- docs/
|   |-- informe_tecnico.md
|-- assets/
|   |-- (texturas/iconos, opcional)
|-- requirements.txt
|-- README.md
```

Razón de esta separación: cada carpeta responde a una sola pregunta. `physics/` responde “¿cómo interactúan los cuerpos?”. `integrators/` responde “¿cómo avanza el tiempo?”. `kepler/` responde “¿se cumplen las leyes?”. `render/` responde “¿qué ve el usuario?”. Si mezclas física con renderizado en un mismo archivo, cualquier bug de gráficos te hace dudar de tus cálculos. Error de principiante. No lo cometas.

4.3. Archivos y Responsabilidades

- `physics/bodies.py`: Define qué es un cuerpo celeste: masa, posición, velocidad, nombre. Es una estructura de datos, no lógica. Aquí no calculas nada, solo guardas el estado.
- `physics/gravity.py`: Calcula la fuerza gravitacional entre cuerpos usando la ley de gravitación universal ($F = G \cdot m_1 \cdot m_2 / r^2$). Recibe posiciones y masas, devuelve fuerzas o aceleraciones. No sabe nada de integración ni de gráficos.
- `physics/conservation.py`: Calcula momento angular y energía mecánica total en cada instante. Se usa para verificar que tu integrador no está introduciendo errores (si estos valores se disparan, tu método numérico está mal implementado).
- `integrators/euler_cromer.py` y `integrators/verlet.py`: Reciben el estado actual (posición, velocidad, aceleración) y el paso de tiempo (dt), devuelven el siguiente estado. Aquí es donde decides cómo avanza la simulación en el tiempo. Implementa ambos: te sirve para comparar precisión en tu informe.
- `kepler/kepler_laws.py`: Verifica las tres leyes: 1) órbitas son elipses con el sol en un foco, 2) área barrida es constante en tiempos iguales, 3) $T^2 \propto a^3$. Toma los datos de posición generados por la simulación y los contrasta contra la teoría.
- `render/scene3d.py`: Dibuja el sistema en 3D: posiciones actuales, trayectorias, ejes. Solo lee el estado, nunca lo modifica.
- `render/plots.py`: Gráficas 2D de apoyo: energía vs tiempo, momento angular vs tiempo, verificación de la segunda ley de Kepler. Estas gráficas son las que sustentan tu análisis, no decoración.
- `ui/controls.py`: Sliders/inputs para masa y velocidad inicial. Traduce la interacción del usuario en cambios de estado inicial. No hace física ni renderizado.
- `main.py`: Orquesta todo: inicializa cuerpos, corre el loop de simulación, llama al render en cada frame. Es el único archivo que conoce todos los módulos.

4.4. Tecnologías

Componente	Librería	Justificación
Cálculos numéricos	NumPy	Vectoriza operaciones de posición/velocidad; sin esto, tu simulación es lenta y el código es más largo.
Integración	Código propio	Euler-Cromer y Verlet son algoritmos simples; usar una librería externa te quita el control y el aprendizaje que el profesor evalúa.

Componente	Librería	Justificación
Visualización 3D interactiva	Plotly (con Dash) o VPython	Plotly: gráficos 3D interactivos en navegador, fácil de rotar/zoom, ideal para presentar resultados. VPython: más simple de programar, pensado específicamente para física, pero visualmente más limitado. Si tu prioridad es “verse profesional” → Plotly. Si tu prioridad es “programar rápido sin pelear con la librería” → VPython.
Gráficas de análisis	Matplotlib	Estándar para gráficas 2D científicas, cero fricción.
Interfaz de controles	Dash (si usas Plotly) o controles nativos de VPython	Evita mezclar dos frameworks de UI distintos.

Descarta Pygame: sirve para gráficos 2D con control de frames, pero implementar 3D ahí es reinventar la rueda sin necesidad.

5. Metodología de Medición y Plan de Pruebas

En este proyecto, la medición es principalmente computacional. No se medirán fuerzas o posiciones con sensores físicos; se medirán variables del estado numérico generado por el simulador y se compararán contra resultados teóricos. Esto hace que la incertidumbre principal sea numérica y no instrumental.

5.1. Variables

Variable	Tipo	Cómo se obtiene	Uso
t	Independiente / control	Contador de pasos $\times dt$	Evolución temporal
m	Entrada	Control del usuario / preset	Define interacción gravitacional
$\vec{r} = (x, y, z)$	Dependiente	Integrador	Trayectoria
$\vec{v} = (v_x, v_y, v_z)$	Dependiente	Integrador	Tipo de órbita
$\vec{a}$	Dependiente	<code>gravity.py</code>	Dinámica
E	Indicador	<code>conservation.py</code>	Conservación
L	Indicador	<code>conservation.py</code>	Conservación
T	Resultado	<code>kepler_laws.py</code>	3. ^a ley de Kepler
$A(\Delta t)$	Resultado	Áreas barridas	2. ^a ley de Kepler

5.2. Escenarios de Prueba

5.3. Checklist de Implementación (Orden de construcción)

1. `bodies.py` — sin esto no tienes nada que simular.
2. `gravity.py` — prueba con 2 cuerpos fijos, verifica el cálculo a mano con un caso

simple (ej. Tierra-Sol) antes de seguir.

3. `euler_cromer.py` — el más simple de los dos integradores. Implementalo primero, corre una órbita circular de 2 cuerpos, confirma visualmente (aunque sea con print de coordenadas) que la órbita se cierra.
4. `conservation.py` — corre en paralelo al paso 3. Verifica que energía y momento angular se mantienen aproximadamente constantes. Si no, tu integrador o tu gravedad están mal.
5. `verlet.py` — implementalo después de confirmar que Euler-Cromer funciona. Compara ambos: Verlet debe conservar energía mejor.
6. `plots.py` — gráficas simples de energía/momento vs tiempo. Esto te da retroalimentación visual inmediata de si el paso 4 está bien.
7. `kepler_laws.py` — solo tiene sentido una vez que tienes órbitas correctas generadas.
8. `scene3d.py` — la visualización 3D va casi al final. Es la parte más vistosa pero la que menos aporta si la física de fondo está mal. No empieces por aquí aunque sea tentador.
9. `controls.py` — última pieza. Conecta la interactividad una vez que el motor de física ya es confiable.
10. Casos de prueba: circular, elíptica, hiperbólica — en ese orden de complejidad.

No inviertas tiempo en 3D bonito antes de tener conservación de energía funcionando. Un simulador que se ve espectacular pero viola la física es una nota reprobada.

5.4. Consideraciones de Diseño Críticas

- **Paso de tiempo (dt):** demasiado grande $\rightarrow$ tu órbita “explota” (los cuerpos se disparan, error numérico visible). Demasiado pequeño $\rightarrow$ simulación lenta. Empieza con un dt pequeño y ajusta según estabilidad, no según velocidad de ejecución.
- **Unidades:** usa unidades astronómicas normalizadas (AU, masas solares, años) en vez de metros/kilogramos/segundos. Los números en SI para sistemas planetarios son tan grandes que introduces errores de precisión de punto flotante innecesarios.
- **Trayectoria hiperbólica:** requiere velocidad inicial mayor a la velocidad de escape. Debes calcular y mostrarle al usuario ese umbral en la interfaz, o los inputs producirán resultados sin sentido físico.
- **Separación cálculo/render:** ya mencionado arriba — no negociable para que puedas correr simulaciones largas sin que el framerate de gráficos te limite la precisión del análisis.
- **Validación de conservación:** define una tolerancia (ej. 1% de variación) y repórtala explícitamente en tu informe. Es el criterio objetivo que demuestra que tu integrador es correcto.

5.5. Incertidumbre y Error

Se reportará el error relativo de una magnitud X mediante:

$$\text{Error} = \frac{|X_{\text{sim}} - X_{\text{ref}}|}{X_{\text{ref}}} \times 100 \% \quad (5)$$

Para conservación se usará principalmente la variación respecto al valor inicial:

$$\Delta X(t) = \frac{|X(t) - X(0)|}{X(0)} \times 100 \% \quad (6)$$

Como meta preliminar se propone $\leq 1 \%$, pero esta tolerancia se debe confirmar después de experimentar con dt y ambos integradores.

6. Resultados Esperados (Hipótesis)

Hipótesis general: si el motor de gravedad y los integradores se implementan correctamente y se selecciona un paso de tiempo suficientemente pequeño, el simulador reproducirá cualitativamente las trayectorias orbitales previstas por la mecánica clásica y presentará errores numéricos controlados.

6.1. Predicciones Específicas

- Con una velocidad tangencial igual o cercana a la velocidad circular, el cuerpo describirá una órbita aproximadamente circular.
- Con una velocidad inicial inferior a la circular pero por debajo de la velocidad de escape, se obtendrá una órbita elíptica.
- Con una velocidad superior a la de escape, se obtendrá una trayectoria hiperbólica abierta.
- La energía mecánica y el momento angular presentarán oscilaciones o variaciones pequeñas, pero no una deriva creciente incompatible con la tolerancia definida.
- Verlet debería presentar una conservación de energía comparable o mejor que Euler-Cromer para un dt apropiadamente seleccionado; la comparación se hará con datos, no por suposición.
- La gráfica de energía relativa vs. tiempo debería permanecer alrededor de un valor cercano a cero en un escenario estable.
- La gráfica de momento angular relativo vs. tiempo debería permanecer aproximadamente constante.
- La tasa de área barrida por unidad de tiempo debería ser aproximadamente constante, sustentando la segunda ley de Kepler.
- Para diferentes órbitas ligadas alrededor del mismo cuerpo central, T^2/a^3 debería agruparse alrededor de una constante.

6.2. Gráficas Previstas

Gráfica	Eje X	Eje Y	Interpretación
Trayectoria orbital	x o tiempo	y / z	Forma circular, elíptica o hiperbólica
Energía relativa	t	$\Delta E/E_0$ (1 %)	Estabilidad del integrador
Momento angular relativo	t	$\Delta L/L_0$ (1 %)	Conservación del momento angular
Área barrida	t	A acumulada o $A/\Delta t$	2. ^a ley de Kepler
T^2 vs a^3	a^3	T^2	3. ^a ley; relación aproximadamente lineal

Resultado de éxito del proyecto: no será únicamente que la animación funcione. El criterio de éxito será demostrar, con pruebas reproducibles, que el modelo produce las órbitas esperadas y que sus magnitudes de conservación y relaciones de Kepler cumplen una tolerancia cuantitativa razonable.

7. Lista de Materiales y Presupuesto

Al ser un proyecto de simulación computacional, el costo físico es bajo. El recurso principal es un computador capaz de ejecutar Python, NumPy y la herramienta de visualización elegida. Se propone VPython como primera opción para la versión interactiva porque está orientado a visualización de fenómenos físicos y permite construir la escena sin añadir una capa web compleja. Plotly/Dash puede evaluarse posteriormente si se prioriza una interfaz web más sofisticada.

Elemento	Cant.	Estado / Origen	Costo estimado (COP)	Observación
Computador de desarrollo	1	Ya disponible	0	Debe ejecutar Python y librerías científicas.
Python 3.x	1	Software libre	0	Entorno de programación.
NumPy	1	Software libre	0	Cálculo vectorial y numérico.
VPython	1	Software libre	0	Visualización 3D interactiva.
Matplotlib	1	Software libre	0	Gráficas científicas 2D.
Git + GitHub	1	Software / Servicio	0	Control de versiones y respaldo.
Almacenamiento / USB (opcional)	1	Por adquirir si se requiere	30.000	Respaldo físico.
Impresión del informe (opcional)	1	Por adquirir si se exige	15.000	Depende de la institución.

Total estimado: 0 COP si el equipo ya cuenta con computador e impresión digi-

tal; hasta aproximadamente 45.000 COP si se consideran respaldo físico e impresión. El presupuesto puede reducirse a cero usando únicamente recursos digitales.

8. Análisis de Riesgos y Seguridad

El proyecto presenta menos riesgos físicos que un prototipo mecánico porque no contempla proyectiles, mecanismos de lanzamiento ni voltajes externos. Los riesgos principales están asociados al trabajo prolongado con computador, pérdida de datos y errores de interpretación del modelo.

Riesgo	Probabilidad	Impacto	Mitigación
Pérdida del código o datos	Media	Alto	Git, GitHub y copias periódicas; commits por funcionalidad.
Resultados físicamente incorrectos	Media	Alto	Pruebas unitarias, cálculo manual de gravedad y validación de conservación.
dt demasiado grande	Alta	Alto	Barrido de dt y criterio de estabilidad; no priorizar FPS sobre precisión.
Confusión de unidades	Media	Alto	Definir unidades internas y documentar conversiones.
Errores por acoplar render y física	Media	Medio	Loop físico independiente; renderizado cada N pasos.
Fatiga / postura frente al PC	Media	Medio	Pausas, postura adecuada y organización del puesto.
Instalación de paquetes inseguros	Baja	Medio	Usar PyPI y repositorios oficiales; fijar versiones en <code>requirements.txt</code> .

8.1. Medidas Críticas

- No ejecutar código descargado de fuentes desconocidas sin revisar su procedencia.
- Mantener una copia versionada del proyecto antes de cambios estructurales.
- No aceptar un resultado como correcto solo porque la trayectoria “se ve bien”; validar energía, momento angular y Kepler.
- Si se realiza una demostración presencial, asegurar cables, conexiones eléctricas del computador y condiciones ergonómicas adecuadas.

9. Cronograma de Actividades (Diagrama de Gantt)

10. Referencias

- [1] Newton, I. (1687). *Philosophiae Naturalis Principia Mathematica*. Base histórica de las leyes del movimiento y gravitación.

- [2] Kepler, J. (1609/1619). Trabajos sobre las leyes del movimiento planetario, base histórica de las tres leyes de Kepler.
- [3] OpenStax. *University Physics, Volume 1*. Capítulos sobre leyes de Newton, gravitación, energía y momento angular.
- [4] NumPy Developers. *NumPy Documentation*. Documentación de referencia para arreglos y operaciones numéricas en Python.
- [5] Matplotlib Development Team. *Matplotlib Documentation*. Referencia para visualización y gráficas científicas.
- [6] VPython Documentation. *VPython / GlowScript documentation*. Referencia para visualización 3D interactiva orientada a física.
- [7] Hairer, E., Lubich, C., & Wanner, G. (2006). *Geometric Numerical Integration*. Springer. Referencia avanzada sobre integración numérica y preservación de estructura.